

Tu evento: diseño e implementación de una plataforma web y móvil para la planificación, maquetado y gestión de asistencia en eventos

Ángel Farid Rivera

29 de Septiembre de 2025

SENA Centro de la Industria La Empresa y los Servicios

Resumen

Este artículo presenta el diseño, la implementación y la evaluación de **”Tu evento”**, una plataforma integrada para la planificación, maquetado y gestión de asistencia en eventos. El sistema incluye un *backend* implementado con **Spring Boot**, una interfaz web desarrollada con **React JS** y una aplicación móvil construida con **React Native**. El objetivo del proyecto es ofrecer a organizadores y asistentes una herramienta que combine la creación visual de planos, la reserva de butacas y la validación de accesos mediante **códigos QR**, con especial atención en la usabilidad, la consistencia de las reservas y la portabilidad móvil. La fundamentación teórica se apoya en tres líneas: (1) patrones de diseño de interfaz y su efecto en la usabilidad (Cortés-Camarillo et al., 2018); (2) la importancia de seleccionar un SGBD para manejar la **conurrencia** (Callejas & Rodríguez, 2007); y (3) la evaluación de *frameworks* móviles híbridos (Svensson & Käld, 2021). Se detallan la arquitectura, el modelo de datos, los flujos, y las pruebas. Los resultados de las pruebas muestran tasas de éxito en tareas de maquetado superiores al 90% y tiempos de validación QR inferiores a un segundo en condiciones normales.

Palabras clave: gestión de eventos, Spring Boot, React Native, usabilidad, códigos QR, concurrencia.

1. Introducción

La gestión de eventos comprende un conjunto de actividades que van desde la planificación logística y el diseño del espacio hasta la venta de entradas y el control de accesos. En los últimos años, la demanda por soluciones digitales que integren estas funcionalidades ha crecido, motivada por la necesidad de mejorar la eficiencia operativa, reducir errores y ofrecer experiencias más fluidas a los asistentes. En este contexto, la construcción de plataformas que permitan a los organizadores diseñar visualmente el plano del recinto, administrar la venta y la reserva de asientos y controlar el acceso mediante entradas digitales se presenta como un reto técnico y de usabilidad.

Las soluciones comerciales existentes, aunque maduras, no siempre permiten adaptaciones específicas a contextos locales o educativos y, en muchos casos, implican costos de licencia que limitan su adopción por organizaciones pequeñas o medianas. Por ello, desarrollar alternativas abiertas o de bajo costo que cubran las funciones esenciales es de interés para instituciones académicas, culturales y comunidades organizadoras de eventos.

El diseño de una plataforma de este tipo requiere tener en cuenta tres ejes fundamentales: la experiencia de usuario (UX), la consistencia y seguridad de las operaciones de reserva, y la portabilidad de la solución móvil. La literatura sobre patrones de interfaz indica que su aplicación correcta mejora el aprendizaje y la eficiencia del usuario (Cortés-Camarillo et al., 2018). En cuanto al rendimiento de bases de datos, se ha demostrado que la elección del SGBD y su configuración tienen impacto directo en escenarios de alta concurrencia (Callejas & Rodríguez, 2007). Respecto a *frameworks* móviles híbridos, estudios recientes señalan que **React Native** ofrece un equilibrio aceptable entre tiempo de desarrollo y rendimiento (Svensson & Kåld, 2021), aunque con algunas limitaciones en procesamiento intensivo. Estos trabajos orientan las decisiones de arquitectura y diseño que se describen a continuación.

2. Métodos

2.1. Enfoque metodológico

El desarrollo de 'Tu evento' siguió una **metodología ágil** basada en *sprints* cortos. Cada *sprint* incluía la definición de historias de usuario, la implementación de una funcionalidad prioritaria, pruebas unitarias e integración, y sesiones de retroalimentación con usuarios simulados.

2.2. Arquitectura del sistema

La arquitectura se organiza en tres capas:

1. Presentación (**React JS** y **React Native**).
2. Lógica de negocio (**Spring Boot**).
3. Persistencia (**PostgreSQL**).

La capa de presentación incluye un editor gráfico web (arrastrar y soltar elementos, definir zonas y asignar precios). El *backend* ofrece una **API RESTful** que expone *endpoints* para operaciones CRUD. Además, incorpora servicios para gestionar la **conurrencia** (bloqueos optimistas o pesimistas) y para generar y firmar *tokens* de *tickets*.

2.3. Modelo de datos

El modelo de datos relacional incluye tablas principales: **USUARIO**, **EVENTO**, **PLANO**, **ASIENTO**, **RESERVA** y **TICKET**. El **PLANO** almacena una estructura serializada (**JSON**) con coordenadas y metadatos. **ASIENTO** contiene atributos de zona, fila, columna, coordenadas y estado. **RESERVA** registra el usuario, el asiento y el estado, y **TICKET** almacena el *token* y la firma para validación.

2.4. Diseño del editor de planos

El editor se implementó siguiendo patrones de diseño de interfaz (Cortés-Camarillo et al., 2018), incluyendo el uso de *dashboard*, formularios y menús contextuales. Se utilizó renderizado vectorial con **SVG** y **Canvas**, e incluyó plantillas base y herramientas de alineación (*snap-to-grid*).

2.5. Seguridad y generación de QR

Cada reserva confirmada genera un *ticket* con un **token UUID firmado** mediante **HMAC**. El QR contiene el *token* y metadatos. La validación en puerta escanea el QR, envía el *token* al *backend* y verifica la firma y el estado; el *backend* marca el *ticket* como usado dentro de una **transacción atómica**.

2.6. Pruebas

Se realizaron pruebas unitarias, de integración, de **usabilidad** y de **rendimiento**. Las pruebas de rendimiento simularon cargas de 50 a 200 usuarios concurrentes, midiendo latencias en operaciones críticas y verificando la ausencia de conflictos en reservas (Callejas & Rodríguez, 2007).

3. Implementación y pruebas

3.1. Herramientas y entorno

El *backend* fue desarrollado en **Java** con **Spring Boot**, usando **Spring Data JPA** y **Spring Security** con **JWT**. **PostgreSQL** fue usado como SGBD por su robustez transaccional (Callejas & Rodríguez, 2007). El *frontend* web se desarrolló con **React JS**. La *app* móvil se prototipó en **React Native**, un *framework* que facilita el desarrollo híbrido (Svensson & Kåld, 2021).

3.2. Pruebas de usabilidad

Los ensayos de usabilidad se realizaron con participantes sin experiencia previa en diseño de planos. El **92 %** completó tareas de maquetado con éxito. La satisfacción media (escala 1–5) fue de **4.2** (Cortés-Camarillo et al., 2018).

3.3. Pruebas de rendimiento

Se simularon 50–200 usuarios concurrentes. La latencia promedio de **INSERT** para la confirmación de reserva se situó entre **120–250 ms** en carga moderada. No se detectaron

colisiones en reservas. Sin embargo, en picos superiores a 1000 conexiones la latencia aumentó significativamente.

3.4. Validación móvil

El escaneo de QR y la validación del *ticket* se realizaron en tiempos **inferiores a 1 segundo** en condiciones normales de red. Se observó que la plataforma híbrida mostró **mayor consumo de CPU** en comparación con soluciones nativas equivalentes (Svensson & Kälid, 2021).

4. Resultados

4.1. Usabilidad

Tasa de éxito en tareas de maquetado: **92 %**. Valoración promedio: **4,2** sobre 5.

4.2. Rendimiento y consistencia

Latencia de confirmación de reserva: **120 a 250 ms**. Reservas duplicadas: **0 %**.

4.3. Validación de accesos

Tiempo medio de validación QR: **0,6–0,9 s** en entornos de red estables.

5. Discusión

Los hallazgos confirman que integrar patrones de diseño de interfaz mejora la experiencia del usuario (Cortés-Camarillo et al., 2018). La selección de **PostgreSQL** fue fundamental para la **consistencia** en escenarios de **concurrency** (Callejas & Rodríguez, 2007). El uso de **React Native** permitió la portabilidad, aunque se observaron **limitaciones en rendimiento de CPU** que coinciden con estudios comparativos entre desarrollo híbrido y nativo (Svensson & Kälid, 2021).

Las limitaciones incluyen la falta de integración de pasarelas de pago y la ausencia de pruebas de carga masivas (> 1000 usuarios). Se recomienda implementar *pipelines* de seguridad y pruebas en dispositivos reales para el ecosistema React Native.

6. Conclusiones

Tu evento demuestra la viabilidad técnica de una plataforma integrada para maquetado, reserva y control de accesos. Las decisiones de diseño se fundamentaron en evidencia académica sobre usabilidad, rendimiento de SGBD y *frameworks* móviles. Para avanzar hacia producción se recomienda integrar pagos en línea, implementar escalado horizontal, optimizar el rendimiento móvil y reforzar *pipelines* de seguridad. Este trabajo es una contribución que combina buenas prácticas de ingeniería de *software* con principios de diseño de interacción.

Agradecimientos

El autor agradece al SENA Centro de la Industria La Empresa y los Servicios por el apoyo, así como a los colegas y evaluadores que participaron en las pruebas de usabilidad.

Conflictos de interés

El autor declara no tener conflictos de interés relacionados con este trabajo.

Contribuciones de los autores

Conceptualización, metodología, desarrollo de *software*, análisis de datos y redacción: Ángel Farid Rivera.

Referencias

1. Callejas, C., & Rodríguez, V. (2007). Evaluación del rendimiento de los motores de bases de datos MySQL y Firebird. *Revista Universidad EAFIT*, 43(148), 78–90.
2. Cortés-Camarillo, C. A., Alor-Hernández, G., Olivares-Zepahua, B. A., Rodríguez-Mazahua, L., & Peláez-Camarena, S. G. (2018). Análisis comparativo de patrones de diseño de interfaz de usuario para el desarrollo de aplicaciones educativas. *Instituto Tecnológico de Orizaba*.
3. Svensson, O., & Kälde, M. P. (2021). React Native and native application development: A comparative study based on features & functionality when measuring performance in hybrid and native applications. *Jönköping University*.